

CodeGuardian AI

Master Product Requirements & Implementation Specification

Document Version: 1.0

Project Type: AI-powered software engineering quality platform

Target: Portfolio / Python Backend Developer placement project

Primary User: Software developers

Secondary User: Engineering leads / reviewers

MVP Language Analyzed: Python

Backend: Django + Django REST Framework

Database: MySQL

Frontend: HTML + CSS + Vanilla JavaScript + Bootstrap 5

AI: Google Gemini API

Deployment Target: Render

Repository: GitHub

1. PROJECT VISION

Build a professional web application called **CodeGuardian AI** that acts as an intelligent first-level code reviewer.

The platform should help developers identify:

- security vulnerabilities
- code quality problems
- maintainability problems
- excessive complexity
- poor coding practices
- potential performance problems
- improvement opportunities

The platform combines:

1. deterministic static code analysis
2. Python AST analysis
3. established code-quality/security tools
4. Google Gemini AI
5. a Django backend

6. a professional dashboard

The objective is NOT to replace human code reviewers.

The objective is:

Catch common problems before the code reaches a human reviewer and explain those problems in a way a developer can understand and fix.

2. REAL-WORLD PROBLEM

AI coding assistants have dramatically increased the amount of code developers can produce.

However, faster code generation can also create:

- insecure code
- duplicated logic
- unnecessarily complex code
- poor naming
- missing error handling
- hardcoded secrets
- inefficient database operations
- code that technically works but is difficult to maintain

Human reviewers then spend significant time identifying these issues manually.

CodeGuardian AI provides an automated first-pass review.

3. PRODUCT OBJECTIVES

The application must allow a user to:

1. create an account
2. log in securely
3. create projects
4. submit Python source code
5. upload Python .py files
6. run static analysis

7. run AI-assisted review
 8. receive a quality report
 9. see issues grouped by severity
 10. understand why each issue matters
 11. receive suggested fixes
 12. view historical reviews
 13. compare previous review results
 14. view project-level quality metrics
-

4. TARGET USERS

4.1 Developer

A developer who wants to check code before submitting it for review.

4.2 Student / Junior Developer

A learner who wants to understand why their code is poor and how to improve it.

4.3 Engineering Reviewer

A reviewer who wants a quick overview of common issues before manually reviewing code.

5. MVP SCOPE

The MVP MUST contain these core capabilities.

Feature 1 — Authentication

Implement:

- registration
- login
- logout
- password hashing using Django's built-in authentication system
- authenticated dashboard
- protected pages/API endpoints

Use Django authentication.

For REST APIs, use JWT authentication with Django REST Framework Simple JWT.

6. FEATURE 2 — PROJECT MANAGEMENT

Authenticated users can create projects.

Project fields:

- project name
- description
- owner
- created date
- updated date

Users can:

- create project
- view project
- edit project
- delete project
- view project review history

A user must only be able to access their own projects.

7. FEATURE 3 — CODE SUBMISSION

Users must be able to submit Python code in two ways.

Method A — Paste code

Provide a large code editor/text area.

Example:

```
def calculate_total(items):  
    total = 0  
    for item in items:  
        total += item["price"]
```

return total

Method B — Upload .py file

Allow Python .py files.

Validate:

- file extension
- MIME type where possible
- file size

Reject unsupported file types.

For MVP, do not allow arbitrary executable files.

8. CRITICAL SECURITY REQUIREMENT

NEVER EXECUTE USER-SUBMITTED CODE.

CodeGuardian AI is a static analysis platform.

Uploaded Python code must only be:

- read
- parsed
- analyzed
- scored

It must never be executed using:

`exec()`

`eval()`

`subprocess`

`os.system()`

or any equivalent mechanism.

This is a fundamental security requirement.

9. FEATURE 4 — STATIC CODE ANALYSIS

The platform must use deterministic analysis before asking Gemini for AI analysis.

Use the following tools where technically appropriate:

Python AST

Use Python's built-in ast module to inspect:

- functions
- classes
- imports
- loops
- conditions
- function length
- nesting
- structural patterns

Ruff

Use Ruff for Python linting and code-quality issues.

Bandit

Use Bandit for common Python security issues.

Radon

Use Radon for code complexity metrics.

The system should gracefully handle cases where a tool cannot analyze the submitted code.

10. STATIC ANALYSIS RESULT

Convert tool output into a common internal issue format.

Each issue should contain:

- issue ID
- category
- severity
- title
- description

- file
- line number if available
- source tool
- rule ID if available
- suggested fix

Example:

```
{  
  "category": "Security",  
  "severity": "Critical",  
  "title": "Possible hardcoded password",  
  "description": "A password appears to be directly stored in source code.",  
  "line_number": 18,  
  "source": "Bandit"  
}
```

11. FEATURE 5 — AI CODE REVIEW

After deterministic analysis, send appropriate code/context to the Gemini API.

Gemini should NOT be responsible for every basic linting rule.

The AI layer should focus on:

- explaining issues
- identifying higher-level problems
- suggesting improvements
- improving readability
- identifying maintainability concerns
- explaining performance concerns
- recommending refactoring approaches

The AI should receive structured information from the static analysis layer when possible.

12. GEMINI RESPONSE REQUIREMENTS

The AI response must be structured JSON rather than uncontrolled plain text.

Expected structure:

```
{
  "summary": "...",
  "overall_assessment": "...",
  "issues": [
    {
      "title": "...",
      "category": "...",
      "severity": "...",
      "line_number": 10,
      "explanation": "...",
      "recommendation": "...",
      "suggested_fix": "..."
    }
  ],
  "strengths": [
    "...",
    "..."
  ],
  "improvements": [
    "...",
    "..."
  ]
}
```

Validate the response before storing it.

If the AI response is invalid or unavailable, the application must still display deterministic static-analysis results.

13. AI PROMPT DESIGN

Create a dedicated backend service for AI interaction.

Do NOT put Gemini API calls directly inside Django views.

Recommended conceptual structure:

AI Service

↓

Prompt Builder

↓

Gemini API

↓

Response Validator

↓

Normalized Review Result

Keep the Gemini API key in environment variables.

Never hardcode API keys in source code.

14. FEATURE 6 — QUALITY SCORE

Every review should generate an overall score.

Use a transparent scoring model.

Suggested categories:

- Security
- Code Quality
- Maintainability
- Complexity
- Performance

- Best Practices

Example:

Security 90

Code Quality 82

Maintainability 76

Complexity 85

Performance 80

Best Practices 88

Overall Score 84

The scoring algorithm should be deterministic and documented.

Do not allow Gemini alone to arbitrarily decide the score.

15. ISSUE SEVERITY

Use four severity levels:

Critical

Potential serious security or correctness problem.

High

Significant security, reliability, or maintainability concern.

Medium

Meaningful quality problem that should be addressed.

Low

Minor improvement or style issue.

Also support:

Info

Positive observation or informational recommendation.

16. FEATURE 7 — REVIEW REPORT

After analysis, display a complete report.

The report must contain:

Summary

Short explanation of the review.

Overall Score

Large visual score.

Category Scores

Show each category.

Issue Summary

Example:

Critical 1

High 3

Medium 7

Low 5

Info 4

Issue List

Each issue should show:

- severity
- category
- title
- line number
- explanation
- recommendation
- source

17. CODE VIEW

When possible, show the submitted code alongside detected issues.

Example:

Line 18

```
password = "admin123"
```

CRITICAL

Hardcoded Secret

Why:

Sensitive credentials should not be stored directly
inside source code.

Fix:

Use environment variables or a secret manager.

The UI should make it easy to understand which part of the code caused the issue.

18. FEATURE 8 — REVIEW HISTORY

Users must be able to view previous reviews.

History should show:

- project
- review date
- overall score
- issue count
- critical issues
- high issues
- status

Users can open an old review.

19. FEATURE 9 — SCORE COMPARISON

For the same project, compare the latest review with the previous review.

Example:

Previous Score: 72

Current Score: 84

Improvement: +12

Show:

- score change
- issue count change
- critical issue change
- high issue change

This demonstrates meaningful product thinking.

20. DASHBOARD

Create a professional dashboard.

Dashboard should contain:

Overview cards

- Total Projects
- Total Reviews
- Average Quality Score
- Critical Issues

Recent Reviews

List recent analysis activity.

Quality Trend

Show score over time using Chart.js.

Issue Distribution

Show severity/category distribution.

Quick Actions

- New Project
 - New Code Review
 - View History
-

21. FRONTEND DESIGN

Use:

- HTML5
- CSS3
- Vanilla JavaScript
- Bootstrap 5
- Chart.js

Do NOT use React.

Do NOT introduce a frontend framework.

Design should be:

- professional
- modern
- responsive
- developer-oriented
- clean
- dashboard-focused

Recommended visual direction:

- dark developer dashboard
- clear cards
- code-editor style areas
- severity badges
- readable typography

- responsive layout

Do not overuse animations.

Functionality is more important than visual effects.

22. DJANGO ARCHITECTURE

Use Django with clearly separated applications.

Recommended apps:

accounts/

projects/

reviews/

analyzer/

ai_engine/

dashboard/

Responsibilities:

accounts

Authentication and user-related functionality.

projects

Project management.

reviews

Review records, issues, reports, scores, and history.

analyzer

Static analysis logic.

ai_engine

Gemini integration and AI prompt/response handling.

dashboard

Dashboard views and analytics.

Keep business logic outside large views.

23. RECOMMENDED BACKEND STRUCTURE

Use a clean structure similar to:

codeguardian/

|

├─ manage.py

|

├─ config/

|

├─ settings.py

|

├─ urls.py

|

├─ asgi.py

|

└─ wsgi.py

|

├─ accounts/

├─ projects/

├─ reviews/

├─ analyzer/

├─ ai_engine/

├─ dashboard/

|

├─ templates/

├─ static/

├─ media/

|

├─ requirements.txt

├─ .env.example

├─ .gitignore

|

└─ README.md

The exact structure may be adjusted if necessary, but maintain clear separation of responsibilities.

24. DATABASE DESIGN

Use MySQL.

Minimum models:

User

Use Django's built-in User model or a properly designed custom user model.

Project

Fields:

- id
- owner
- name
- description
- created_at
- updated_at

CodeSubmission

Fields:

- id
- project
- source_code
- file_name
- submission_type
- created_at

Review

Fields:

- id
- project

- code_submission
- overall_score
- security_score
- quality_score
- maintainability_score
- complexity_score
- performance_score
- best_practices_score
- summary
- created_at

ReviewIssue

Fields:

- id
- review
- category
- severity
- title
- description
- recommendation
- suggested_fix
- line_number
- source
- rule_id
- created_at

AIAnalysis

Fields:

- id
- review

- model_name
- prompt_version
- response_status
- raw_response if necessary
- created_at

Do not store sensitive API keys.

25. DATABASE RELATIONSHIPS

Conceptually:

User

|

└─< Project

|

└─< CodeSubmission

|

└─< Review

|

└─< ReviewIssue

|

└─ AIAnalysis

A user owns many projects.

A project has many submissions.

A project has many reviews.

A review has many issues.

A review may have one or more AI analysis records depending on implementation.

26. REST API

Use Django REST Framework.

Minimum endpoints:

Authentication

POST /api/auth/register/

POST /api/auth/login/

POST /api/auth/refresh/

Projects

GET /api/projects/

POST /api/projects/

GET /api/projects/<id>/

PUT /api/projects/<id>/

DELETE /api/projects/<id>/

Code Review

POST /api/projects/<id>/reviews/

GET /api/projects/<id>/reviews/

GET /api/reviews/<id>/

Submission

POST /api/projects/<id>/submissions/

Dashboard

GET /api/dashboard/

GET /api/projects/<id>/analytics/

The exact URL design may be adjusted to follow REST best practices.

27. API BEHAVIOR

Use correct HTTP status codes.

Examples:

200 OK

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Validate all incoming data.

Never trust client-side validation alone.

28. ERROR HANDLING

The application must gracefully handle:

- invalid Python syntax
- empty code
- oversized files
- unsupported file types
- static-analysis failures
- Gemini API failure
- invalid Gemini response
- database errors
- authentication errors
- network/API timeout

Do not expose internal stack traces to normal users.

Display useful user-friendly error messages.

Log technical errors on the backend.

29. GEMINI FAILURE FALLBACK

If Gemini is unavailable:

The review should NOT completely fail.

The application should:

1. complete deterministic analysis

2. store the review
3. indicate that AI analysis was unavailable
4. show static-analysis findings
5. allow the user to retry AI analysis later

This is an important reliability requirement.

30. SECURITY REQUIREMENTS

Implement:

- Django password hashing
- JWT authentication for APIs
- CSRF protection where applicable
- authorization checks
- input validation
- file validation
- upload size limits
- secure environment variables
- no API keys in Git
- no execution of uploaded code
- safe error handling
- user-level data isolation

A user must never be able to access another user's projects or reviews by changing an ID in the URL/API request.

31. FILE UPLOAD SECURITY

Only allow:

.py

For MVP.

Set a reasonable maximum file size.

Reject:

.exe

.bat

.sh

.ps1

and other executable formats.

Never execute uploaded files.

Store uploads safely.

32. AI COST CONTROL

Do not unnecessarily send huge files to Gemini.

Before AI analysis:

1. inspect file size
2. extract relevant information
3. use static analysis results
4. truncate or reject excessively large submissions where necessary

Display a useful message when code exceeds the supported analysis size.

33. API KEY MANAGEMENT

Use .env.

Example:

SECRET_KEY=your-secret-key

DEBUG=True

GEMINI_API_KEY=your-gemini-key

DB_NAME=codeguardian

DB_USER=root

DB_PASSWORD=your-password

DB_HOST=localhost

DB_PORT=3306

Create:

.env.example

Do NOT commit .env.

Add .env to .gitignore.

34. TESTING

Implement meaningful tests.

At minimum test:

Authentication

- registration
- login
- protected endpoint

Projects

- create
- retrieve
- update
- delete
- authorization

Code submission

- valid Python
- invalid Python
- invalid file
- oversized file

Analyzer

- AST parsing
- analyzer failure handling

- issue normalization

Review

- score generation
- issue storage
- history

AI

Mock Gemini API calls.

Do not depend on the real Gemini API for automated unit tests.

35. LOGGING

Implement Django/Python logging.

Log:

- review creation
- analysis failures
- AI failures
- authentication errors
- unexpected backend errors

Do not log:

- passwords
 - API keys
 - sensitive source code unnecessarily
-

36. PERFORMANCE

For MVP:

- use Django ORM efficiently
- avoid unnecessary database queries
- use `select_related` / `prefetch_related` where appropriate
- paginate review history

- avoid sending unnecessary data to Gemini

Do not introduce Redis, Celery, Kafka, Elasticsearch, or microservices unless genuinely required.

The MVP should remain understandable to a junior Python developer.

37. WHAT NOT TO BUILD IN MVP

Do NOT build these initially:

- GitHub OAuth integration
- GitHub webhooks
- Pull Request bot
- VS Code extension
- Slack integration
- multi-language support
- Kubernetes
- microservices
- advanced distributed systems
- custom machine-learning model
- custom LLM
- vector database
- complex DevOps infrastructure
- payment system

Record these as future features.

38. FUTURE VERSION

After MVP, possible upgrades include:

Version 2

- GitHub repository integration
- Pull Request analysis

- automatic review comments
- team accounts
- team analytics
- multiple Python projects
- downloadable PDF reports
- email notifications

Version 3

- JavaScript support
- Java support
- C++ support
- CI/CD integration
- GitHub Actions
- VS Code extension
- organization-level dashboards

These are NOT required for the placement MVP.

39. PRODUCT WORKFLOW

Main user workflow:

Register

↓

Login

↓

Dashboard

↓

Create Project

↓

Submit Python Code

↓

Validate Code



Static Analysis



Bandit / Ruff / Radon / AST



Normalize Findings



Gemini AI Analysis



Combine Results



Calculate Scores



Save Review



Display Report



User Fixes Code



Submit Again



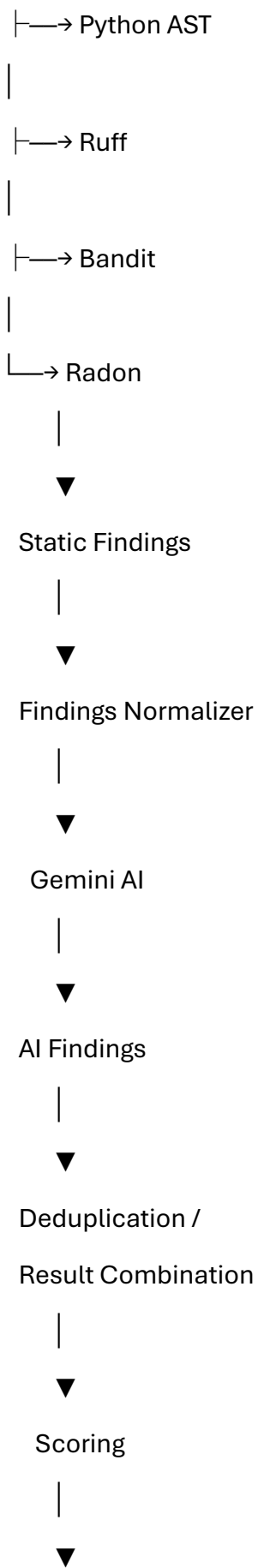
Compare Results

40. REVIEW ENGINE DESIGN

The review engine should conceptually work as:

Code





Review Report

Keep this architecture modular so analyzer components can be replaced or extended later.

41. SCORING LOGIC

Create a transparent scoring service.

Example conceptual approach:

Start at 100

Critical issue → major deduction

High issue → significant deduction

Medium issue → moderate deduction

Low issue → small deduction

Apply reasonable caps so one category cannot produce nonsensical scores.

The exact numerical weights can be selected during implementation but **MUST** be documented in code and README.

AI-generated observations should not arbitrarily destroy the score.

42. DUPLICATE ISSUE HANDLING

If Bandit and Gemini identify the same underlying issue:

Do not display two identical issues.

Normalize and deduplicate where practical.

The final issue should indicate relevant sources when appropriate.

Example:

Sources:

Bandit + AI Review

43. UI PAGES

Build the following pages.

Public

/

/about

/login

/register

Authenticated

/dashboard

/projects

/projects/create

/projects/<id>

/projects/<id>/review

/reviews/<id>

/history

/profile

The exact URL structure can be adjusted if needed.

44. LANDING PAGE

The landing page should clearly explain:

Hero

CodeGuardian AI

Intelligent first-level code review for modern development teams.

CTA:

Analyze Your Code

Secondary CTA:

View Demo

Sections:

- Problem
- How it works

- Features
- Example review
- Security
- Technology
- Future roadmap

Do not make unsupported claims such as "99% accurate."

45. REVIEW PAGE

The review page is the most important UI.

Recommended structure:

CodeGuardian AI

Project: Django API

Overall Score

84 / 100

Security 90

Quality 82

Maintainability 76

Complexity 85

Critical 1

High 3

Medium 7

Low 5

Issues

[CRITICAL] Hardcoded Secret

Line 18

Why:

...

Recommendation:

...

Suggested Fix:

...

46. DASHBOARD ANALYTICS

Use Chart.js.

Charts:

1. Quality score over time
2. Issues by severity
3. Issues by category

Keep charts simple and readable.

47. README REQUIREMENTS

Generate a professional README containing:

Project Overview

Problem

Solution

Features

Architecture

Tech Stack

Installation

Environment Variables

Database Setup

Running Locally

API Documentation

Testing

Security

Screenshots

Future Roadmap

Lessons Learned

Author

Do not claim technologies/features that are not actually implemented.

48. API DOCUMENTATION

Use DRF-compatible OpenAPI documentation if practical.

Document:

- endpoint
- method
- authentication

- request body
- response
- status codes
- errors

Swagger/OpenAPI may be added after the core API is functional.

49. GIT REQUIREMENTS

Use Git from the beginning.

Recommended commits:

Initial Django setup

Add authentication

Add project management

Add code submission

Add static analyzer

Add AI review engine

Add review reports

Add dashboard

Add testing

Prepare deployment

Do not make one giant final commit.

50. ENVIRONMENT REQUIREMENTS

Development environment:

- Python 3
- Django
- Django REST Framework
- MySQL
- Git

- VS Code
- Postman
- Gemini API key

Use a Python virtual environment.

Generate:

requirements.txt

51. DEPLOYMENT READINESS

The application should be structured so it can later be deployed to Render.

Prepare:

- production settings
- environment variables
- requirements.txt
- Gunicorn
- WhiteNoise where appropriate
- static file configuration
- secure secret configuration
- DEBUG controlled by environment variable

Deployment is NOT part of the initial local implementation phase.

52. ACCEPTANCE CRITERIA

The project is considered MVP-complete only when all of the following work:

- User can register
- User can log in
- User can access protected dashboard
- User can create a project
- User can submit Python code
- User can upload .py files

- Invalid Python is handled correctly
- Uploaded code is never executed
- AST analysis works
- Ruff analysis works
- Bandit analysis works
- Radon analysis works
- Static findings are normalized
- Gemini integration works
- Gemini response is validated
- AI failure does not destroy the entire review
- Overall score is generated
- Category scores are generated
- Issues are displayed by severity
- Review is stored
- Review history works
- Previous and current reviews can be compared
- Dashboard analytics work
- Users cannot access other users' data
- API authentication works
- Basic automated tests exist
- .env is excluded from Git
- README is complete
- Application works reliably locally

53. DEVELOPMENT PRIORITY

Implement in this exact priority.

Priority 1 — Foundation

- Django project

- MySQL
- authentication
- project models
- basic dashboard

Priority 2 — Code Submission

- paste code
- file upload
- validation
- storage

Priority 3 — Static Analysis

- AST
- Ruff
- Bandit
- Radon
- normalized issue format

Priority 4 — AI

- Gemini service
- prompt builder
- structured response
- response validation
- fallback behavior

Priority 5 — Reports

- scoring
- issue display
- review history
- comparison

Priority 6 — UI Polish

- dashboard

- charts
- responsive design
- loading states
- error states

Priority 7 — Quality

- tests
- security review
- README
- API documentation
- Git cleanup

Priority 8 — Deployment Preparation

- production settings
- Unicorn
- static files
- environment variables
- Render readiness

54. IMPLEMENTATION RULES FOR THE CODING AGENT

Build the application as a real maintainable software project.

Do NOT:

- put all logic in views.py
- hardcode API keys
- execute submitted code
- use fake AI responses
- create fake analysis results
- create unnecessary technologies
- create unnecessary microservices
- use React

- use MongoDB
- replace MySQL
- introduce technologies without a clear reason

Prefer:

- Django best practices
- reusable services
- clean models
- serializers
- permissions
- validation
- modular analysis engines
- clear naming
- meaningful comments
- error handling
- tests

55. TECHNOLOGY CONSTRAINTS

The implementation MUST use:

Python

Django

Django REST Framework

MySQL

HTML

CSS

JavaScript

Bootstrap

Chart.js

Google Gemini API

Use:

Python AST

Ruff

Bandit

Radon

where appropriate.

Do NOT introduce:

React

Node.js backend

MongoDB

FastAPI

Flask

Kubernetes

Kafka

RabbitMQ

Elasticsearch

Vector database

unless a later explicit requirement changes the architecture.

56. IMPORTANT PRODUCT PRINCIPLE

CodeGuardian AI is NOT simply:

"Paste code → Gemini says whether it is good."

It must demonstrate the following engineering architecture:

Deterministic Analysis

+

AI Reasoning

+

Normalized Findings

+

Scoring

+

Historical Tracking

=

Engineering Quality Platform

This distinction is important.

The project should demonstrate that AI is being used **where it adds value**, while deterministic tools are used where deterministic rules are better.

57. EXPECTED INTERVIEW DISCUSSION

The finished project should allow the developer to explain:

Django

- Why Django?
- Django app structure
- ORM
- middleware
- authentication
- permissions

DRF

- serializers
- API views
- authentication
- JWT
- HTTP status codes

MySQL

- relationships
- indexes

- normalization
- query optimization

AI

- Gemini API integration
- prompts
- structured outputs
- failure handling
- token/context limitations

Code Analysis

- AST
- static analysis
- Bandit
- Ruff
- Radon
- cyclomatic complexity

Security

- uploaded code safety
- API key security
- authentication
- authorization
- file validation
- data isolation

Architecture

- separation of concerns
- service layer
- analyzer pipeline
- deterministic vs AI analysis

Deployment

- environment variables
- Gunicorn
- static files
- Render

The implementation should make these concepts genuinely understandable rather than adding technologies only for resume keywords.

58. FINAL PRODUCT DEFINITION

At completion, CodeGuardian AI should be a working web application where a developer can:

Create Account



Create Project



Submit Python Code



Run Review



Static Analysis



AI Analysis



Receive Quality Report



Understand Problems



Fix Code



Submit Again

↓

Compare Improvement

The final product should feel like **Version 1.0 of a real developer tool**, not a classroom CRUD application.

The MVP must prioritize:

Reliability > understandable architecture > meaningful functionality > visual polish > extra features.

59. FINAL INSTRUCTION TO THE IMPLEMENTATION AGENT

Using this document as the single source of truth, build the CodeGuardian AI MVP from the ground up.

Before implementing a feature, verify that it is consistent with this specification.

If a technical implementation detail is unspecified, choose the simplest production-appropriate solution that fits the existing Django + MySQL architecture.

Do not expand the scope unnecessarily.

Do not replace the specified technology stack.

Do not create fake functionality.

Every major feature must actually work.

The final application must be runnable locally, testable, documented, and prepared for later deployment to Render.

Build the project in a modular way so future versions can add GitHub integration, Pull Request reviews, team analytics, and additional programming languages without requiring a complete rewrite.